

Greedy Method

1. Fractional Knapsack Problem

Algorithm

1. Calculate value/weight ratio.
2. Sort items in decreasing order.
3. Take full or fractional items.

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

struct Item { int value, weight; };

bool cmp(Item a, Item b) {
    return (double)a.value/a.weight > (double)b.value/b.weight;
}

int main() {
    Item arr[] = {{60,10},{100,20},{120,30}};
    int W = 50;
    int n = 3;

    sort(arr, arr+n, cmp);
    double total = 0;

    for(int i=0;i<n;i++) {
        if(W >= arr[i].weight) {
            W -= arr[i].weight;
            total += arr[i].value;
        } else {
            total += arr[i].value * ((double)W/arr[i].weight);
            break;
        }
    }

    cout << "Maximum value = " << total;
}
```

Time Complexity: $O(n \log n)$

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

 Code - OOPSc++

```
PS C:\Programming languages> cd "c:\Programming languages\OOPSc++\" ; if ($?) { g++ class1.cpp -o class1 } ; if ($?) { .\class1 }
Maximum value = 240
```

2. Job Sequencing with Deadlines

```
#include <bits/stdc++.h>
using namespace std;

struct Job { char id; int deadline, profit; };

bool cmp(Job a, Job b) { return a.profit > b.profit; }

int main() {
    Job arr[] = {{ 'A', 2, 100 }, { 'B', 1, 19 }, { 'C', 2, 27 }, { 'D', 1, 25 } };
    int n=4;
    sort(arr, arr+n, cmp);

    int slot[3]={0};
    for(int i=0; i<n; i++) {
        for(int j=arr[i].deadline-1; j>=0; j--) {
            if(slot[j]==0) {
                slot[j]=1;
                cout<<arr[i].id<<" ";
                break;
            }
        }
    }
}
```

Time Complexity: $O(n^2)$

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

● PS C:\Programming languages> cd "c:\Programming languages\OOPSc++\" ; if (\$?) { g++ class1.cpp -o class1 } ;
A C

3. Kruskal's Algorithm

```
#include <bits/stdc++.h>
using namespace std;

struct Edge { int u,v,w; };

bool cmp(Edge a, Edge b){ return a.w < b.w; }

int parent[100];

int find(int i){ return (parent[i]==i)?i:parent[i]=find(parent[i]); }

void unite(int x,int y){ parent[find(x)] = find(y); }

int main(){
    Edge e[]={ {0,1,10},{0,2,6},{0,3,5},{1,3,15},{2,3,4} };
    int n=4,m=5,cost=0;

    for(int i=0;i<n;i++) parent[i]=i;
    sort(e,e+m,cmp);

    for(int i=0;i<m;i++){
        if(find(e[i].u)!=find(e[i].v)){
            unite(e[i].u,e[i].v);
            cost+=e[i].w;
        }
    }
    cout<<"MST cost="<<cost;
}
```

Time Complexity: $O(E \log E)$

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Programming languages> cd "c:\Programming languages\OOPSc++\" ; if ($?) { g++ class1.cpp -o class1 } ;
MST cost=19
```

4. Prim's Algorithm

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    int graph[5][5] = {
        {0,2,0,6,0},
        {2,0,3,8,5},
        {0,3,0,0,7},
        {6,8,0,0,9},
        {0,5,7,9,0}
    };

    int key[5], parent[5];
    bool mstSet[5];

    for(int i=0;i<5;i++) key[i]=INT_MAX, mstSet[i]=false;

    key[0]=0;
    parent[0]=-1;

    for(int count=0;count<4;count++){
        int min=INT_MAX,u;
        for(int v=0;v<5;v++){
            if(!mstSet[v] && key[v]<min)
                min=key[v],u=v;

            mstSet[u]=true;

            for(int v=0;v<5;v++){
                if(graph[u][v] && !mstSet[v] && graph[u][v]<key[v])
                    parent[v]=u, key[v]=graph[u][v];
            }
        }

        int cost=0;
        for(int i=1;i<5;i++) cost+=graph[i][parent[i]];
        cout<<"Minimum cost="<<cost;
    }
}
```

Time Complexity: $O(V^2)$

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Programming languages> cd "c:\Programming languages\OOPSc++\" ; if ($?) { g++ class1.cpp -o class1 } ;
Minimum cost=16
```

5. Dijkstra Algorithm

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    int graph[5][5] = {
        {0,10,0,5,0},
        {0,0,1,2,0},
        {0,0,0,0,4},
        {0,3,9,0,2},
        {7,0,6,0,0}
    };

    int dist[5]; bool visited[5]={0};

    for(int i=0;i<5;i++) dist[i]=INT_MAX;
    dist[0]=0;

    for(int count=0;count<4;count++){
        int u,min=INT_MAX;
        for(int i=0;i<5;i++)
            if(!visited[i] && dist[i]<min)
                min=dist[i],u=i;

        visited[u]=true;

        for(int v=0;v<5;v++)
            if(graph[u][v] && dist[u]+graph[u][v]<dist[v])
                dist[v]=dist[u]+graph[u][v];
    }

    for(int i=0;i<5;i++) cout<<dist[i]<<" ";
}
```

Time Complexity: $O(V^2)$

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Programming languages> cd "c:\Programming languages\OOPSc++\" ; if ($?) { g++ class1.cpp -o class1 } ;
0 8 9 5 7
```

Dynamic Programming

6. 0/1 Knapsack

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    int W=50;
    int wt[]={10,20,30};
    int val[]={60,100,120};
    int n=3;

    int dp[n+1][W+1];

    for(int i=0;i<=n;i++){
        for(int w=0;w<=W;w++){
            if(i==0||w==0) dp[i][w]=0;
            else if(wt[i-1]<=w)
                dp[i][w]=max(val[i-1]+dp[i-1][w-wt[i-1]], dp[i-1][w]);
            else dp[i][w]=dp[i-1][w];
        }
    }

    cout<<"Max value="<<dp[n][W];
}
```

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Programming languages> cd "c:\Programming languages\OOPSc++\" ; if ($?) { g++ class1.cpp -o class1 } ;
Max value=220
```

7. OBST

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    int freq[]={34,8,50};
    int n=3;
    int cost[n][n];

    for(int i=0;i<n;i++) cost[i][i]=freq[i];

    for(int L=2;L<=n;L++){
        for(int i=0;i<=n-L;i++){
            int j=i+L-1;
            cost[i][j]=INT_MAX;
            int sum=0;
            for(int k=i;k<=j;k++) sum+=freq[k];

            for(int r=i;r<=j;r++){
                int c = ((r>i)?cost[i][r-1]:0) + ((r<j)?cost[r+1][j]:0) +
sum;
                cost[i][j]=min(cost[i][j],c);
            }
        }
    }

    cout<<"Cost="<<cost[0][n-1];
}
```

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Programming languages> cd "c:\Programming languages\OOPSc++\" ; if ($?) { g++ class1.cpp -o class1 } ;
Cost=142
```

8. Traveling Salesperson (DP)

```
#include <bits/stdc++.h>
using namespace std;

int tsp(int mask, int pos, int
n, vector<vector<int>>&dp, vector<vector<int>>&dist) {
    if (mask == (1<<n)-1) return dist[pos][0];
    if (dp[mask][pos] != -1) return dp[mask][pos];

    int ans = INT_MAX;
    for (int city = 0; city < n; city++) {
        if ((mask & (1<<city)) == 0) {
            int newAns = dist[pos][city] +
tsp(mask | 1<<city, city, n, dp, dist);
            ans = min(ans, newAns);
        }
    }
    return dp[mask][pos] = ans;
}

int main() {
    int n = 4;
    vector<vector<int>>
dist = {{0, 10, 15, 20}, {10, 0, 35, 25}, {15, 35, 0, 30}, {20, 25, 30, 0}};
    vector<vector<int>> dp(1<<n, vector<int>(n, -1));

    cout<<"Minimum cost="<<tsp(1, 0, n, dp, dist);
}
```

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Programming languages> cd "c:\Programming languages\OOPSc++\" ; if ($?) { g++ class1.cpp -o class1 } ;
Minimum cost=80
```

Backtracking

9. 8-Queens

```
#include <bits/stdc++.h>
using namespace std;

int board[8];
int total = 0;
bool printed = false;

// Check if safe
bool safe(int row, int col){
    for(int i=0;i<row;i++){
        if(board[i]==col || abs(board[i]-col)==abs(i-row))
            return false;
    }
    return true;
}

// Print board (only once)
void printBoard(){
    for(int i=0;i<8;i++){
        for(int j=0;j<8;j++){
            if(board[i]==j)
                cout << "Q ";
            else
                cout << "0 ";
        }
        cout << "\n";
    }
    cout << "\n";
}

// Backtracking
void solve(int row){
    if(row==8){
        total++;

        // Print only first solution
        if(!printed){
            cout << "One Solution:\n";
            printBoard();
            printed = true;
        }
        return;
    }

    for(int col=0;col<8;col++){
        if(safe(row,col)){
            board[row]=col;
            solve(row+1);
        }
    }
}

int main(){
```

```
    solve(0);  
    cout << "Total solutions = " << total;  
}
```

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Programming languages> cd "c:\Programming languages\OOPSc++\" ; if ($?) { g++ class1.cpp -o class1 } ;  
One Solution:  
Q 0 0 0 0 0 0 0  
0 0 0 0 Q 0 0 0  
0 0 0 0 0 0 0 Q  
0 0 0 0 0 Q 0 0  
0 0 Q 0 0 0 0 0  
0 0 0 0 0 0 Q 0  
0 Q 0 0 0 0 0 0  
0 0 0 Q 0 0 0 0  
  
Total solutions = 92
```

10. Sum of Subsets

```
#include <bits/stdc++.h>
using namespace std;

int arr[]={10,7,5,18,12,20,15}; // renamed from set → arr
int n=7, target=35;
int total = 0;
bool printed = false;

void subset(int sum, int i, vector<int> res){
    if(sum == target){
        total++;

        if(!printed){
            cout << "One Subset: ";
            for(int x : res) cout << x << " ";
            cout << "\n";
            printed = true;
        }
        return;
    }

    if(i == n || sum > target) return;

    // include element
    res.push_back(arr[i]);
    subset(sum + arr[i], i + 1, res);

    // exclude element
    res.pop_back();
    subset(sum, i + 1, res);
}

int main(){
    subset(0, 0, {});
    cout << "Total subsets = " << total;
}
```

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Programming languages> cd "c:\Programming languages\OOPSc++\" ; if ($?) { g++ class1.cpp -o class1 } ;
One Subset: 10 7 18
Total subsets = 4
```

11. Graph Coloring

```
#include <bits/stdc++.h>
using namespace std;

int graph[4][4]={0,1,1,1},{1,0,1,0},{1,1,0,1},{1,0,1,0}};
int color[4];

bool safe(int v,int c){
    for(int i=0;i<4;i++){
        if(graph[v][i] && color[i]==c) return false;
    }
    return true;
}

bool solve(int v){
    if(v==4) return true;

    for(int c=1;c<=3;c++){
        if(safe(v,c)){
            color[v]=c;
            if(solve(v+1)) return true;
            color[v]=0;
        }
    }
    return false;
}

int main(){
    if(solve(0)){
        for(int i=0;i<4;i++) cout<<color[i]<<" ";
    }
}
```

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Programming languages> cd "c:\Programming languages\OOPSc++\" ; if ($?) { g++ class1.cpp -o class1 } ;
1 2 3 2
```

12. Hamiltonian Cycle

```
#include <bits/stdc++.h>
using namespace std;

int
graph[5][5]={0,1,0,1,0},{1,0,1,1,1},{0,1,0,0,1},{1,1,0,0,1},{0,1,1,1,0}};
int path[5];

bool safe(int v,int pos){
    if(!graph[path[pos-1]][v]) return false;
    for(int i=0;i<pos;i++) if(path[i]==v) return false;
    return true;
}

bool solve(int pos){
    if(pos==5) return graph[path[pos-1]][path[0]];

    for(int v=1;v<5;v++){
        if(safe(v,pos)){
            path[pos]=v;
            if(solve(pos+1)) return true;
            path[pos]=-1;
        }
    }
    return false;
}

int main(){
    path[0]=0;
    if(solve(1)){
        for(int i=0;i<5;i++) cout<<path[i]<<" ";
        cout<<path[0];
    }
}
```

Output :

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Programming languages> cd "c:\Programming languages\OOPSc++\" ; if ($?) { g++ class1.cpp -o class1 } ;
0 1 2 4 3 0
```

End of Practical File